

RELATORIO DE DIFICULDADES

Como qualquer projeto que é proposto, acredito que a primeira dificuldade apresentada é o entendimento do que se trata o trabalho. Considero que exista uma falta de interpretação de texto, mas acredito que vai um pouquinho além disso, uma vez que são tecnologias desconhecidas ou diferentes fatores que mudam o entendimento da atividade.

A atividade demandou um esforço muito grande na escolha da API pública, pois elas possuem limitações que dariam um trabalho a mais. A ideia inicial era ter algo que tivesse significado e impacto na hora de ser apresentado. Pensamos em algo da indústria alimentícia que mostrasse para a pessoa se o alimento tinha apresentava alguma restrição a sua alimentação como : intolerância à lactose, glúten, entre outros.

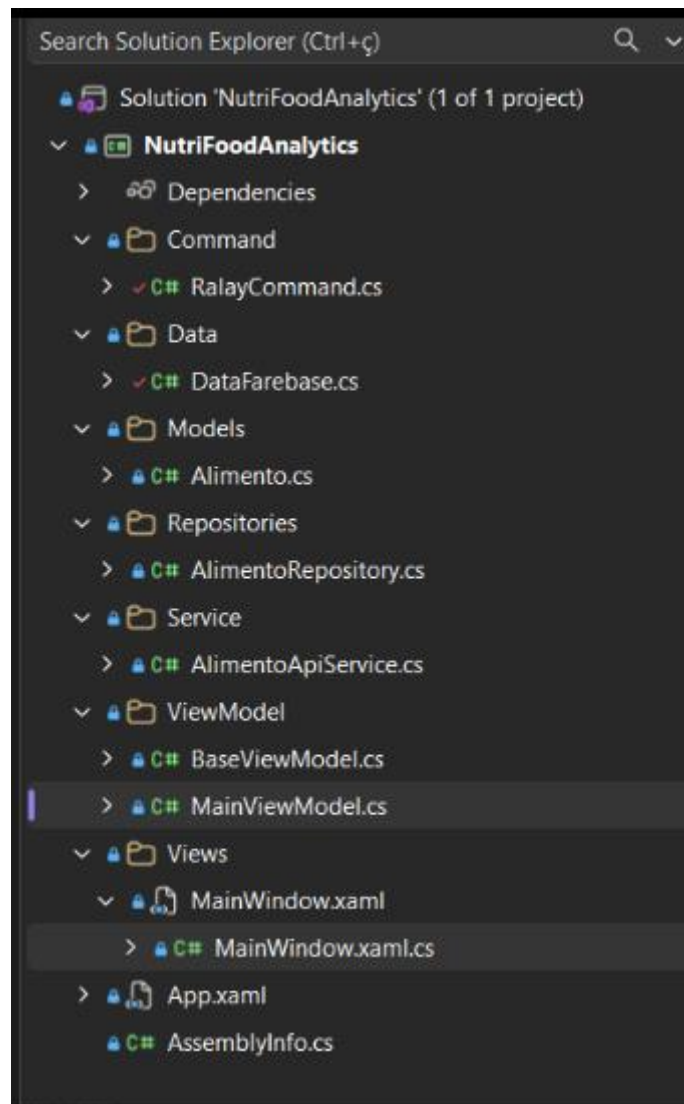
Então pensamos: por que não criar a nossa própria API? Foi aí que começamos a ter problemas, pois gastamos tempo desnecessário para criar algo que já estava pronto. Porém, com mais tempo, acredito sim que seria a melhor escolha, pois acredito que, quando se trabalha com dados gerados por você mesmo, a possibilidade de falhas é menor.

Mesmo assim, seguimos com uma pesquisa de APIs, o que demandou praticamente o período inteiro da aula , e realizamos a votação da divisão das tarefas no Planner Power.

No dia seguinte, começamos a trabalhar nas aplicações. Eu optei pela parte três, que era o retorno ao usuário, e foi a parte mais fora da curva para mim, pois, com base nos meus conhecimentos, sou péssima em interface. Dessa forma, dei início ao desenvolvimento, mas antes fiz uma pesquisa em cima de uma aplicação de uma colega de curso, na qual entendi que seria muito mais difícil do que eu imaginava.



Dessa forma, segui tentando entender a infraestrutura e a regra de negócio. Percebi que, dependendo da modelagem, certas coisas não mudam, como a base ViewModel e RelayCommand. O problema era todo o resto do código.

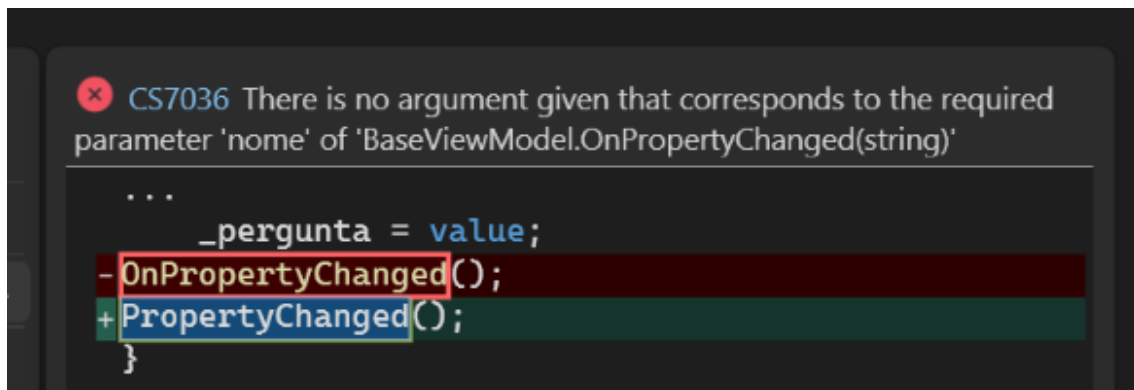


De cara, a minha primeira dificuldade foi entender como modelar sem ter a conexão pronta. Foram feitas telas que seriam alteradas a todo momento, uma vez que a aplicação foi dividida em três partes, e as três tinham que se comunicar com a segunda parte.

Após a criação das pastas, das classes e a instalação dos pacotes, vieram as dificuldades. Afinal de contas, o que eu iria retornar para o usuário? Eu pensei em não fazer somente uma tela convencional, porque o que de fato retorna ao usuário e o que salva é o básico bem feito, que também funciona.

Após uma pesquisa com meu digníssimo professor, descobri que existia a possibilidade de realizar uma pesquisa aberta, na qual o usuário digita um texto, o sistema entende que o que foi digitado está dentro do banco de dados e retorna os dados em tela utilizando o `ObservableCollection`, no qual o texto é dividido em um array na qual recebia a pergunta do usuário armazenava e retornava a resposta depois a pesquisa do

array. Mas já tive problemas pois a maneira que implementei a classe da base view model deu erro ao receber o ObservableCollection,

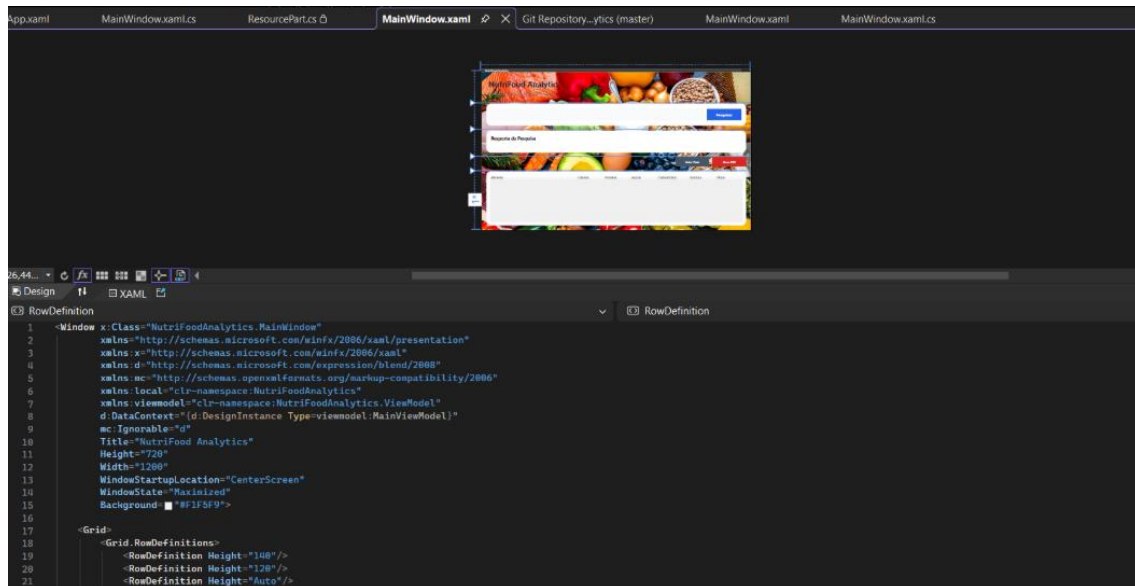


Os problemas encontrados começaram quando a pesquisa era realizada. O sistema entendia que tudo que era carboidrato deveria ser retornado em tela, o que quebrava a lógica da pesquisa personalizada por alimento. Dessa forma, tive que criar uma condicional com `if` e `else` para tratar os casos em que qualquer atributo aparecesse dentro da pesquisa e fosse retornado corretamente.

The image shows a screenshot of a web application titled "NutriFood Analytics". The header has the title and a subtitle "Aplicação web para análise nutricional". Below the header is a search bar with the text "quantos carboidratos possui a banana" and a blue button labeled "Pesquisar". Below the search bar is a section titled "Resposta da Pesquisa" with the text "3 resultado(s) encontrado(s)". Below this section are two buttons: "Listar Tudo" and "Gerar PDF". Below the buttons is a table with the following data:

Alimento	Calorias	Proteína	Açúcar	Carboidratos	Gordura	Fibras
chia seeds	0	0	0	42.3	30.9	34
banana	0	0	12.3	23.2	0.3	2.6
arroz	0	0	0.1	28.1	0.3	0.4

E o que falar sobre a interface? Primeiramente, ela rodou, mas fazer as divisões das linhas dentro da página foi uma dificuldade muito grande. Até mesmo colocar a imagem, que seria algo simples, foi uma luta, pois, na posição em que coloquei inicialmente, ela ficou dentro do Git e funcionava somente em uma sessão do código. Até entender o motivo, foi bem difícil e foram necessárias várias tentativas e solução foi colocar dentro da `grid rowdefinitions`.



E após as mudanças realizadas a imagem finalmente preencheu a tela toda da aplicação após realizado o tratamento das propriedades para deixá-la responsiva conforme a aplicação dentro da pasta de imagens que foi criada dentro do projeto build action resource e copy to output directory Do not copy .

NutriFood Analytics

Sistema inteligente para análise nutricional

quantos carboidratos possui a banana

Pesquisar

Resposta da Pesquisa

3 resultado(s) encontrado(s).

Listar Tudo Gerar PDF

Alimento	Calorias	Proteína	Açúcar	Carboidratos	Gordura	Fibras
chia seeds	0	0	0	42.3	30.9	34
banana	0	0	12.3	23.2	0.3	2.6
arroz	0	0	0.1	28.1	0.3	0.4

E com relação ao PDF tive uma certa dificuldade em relação a estrutura utilizada uma vez que foi necessário alteração e instalação de pacotes como QuestPDF.Fluent.Document com a junção da `AutoGenerateColumns="True"` para que se qualquer nova alteração ocorre as linhas e colunas serem criadas sozinhas sem tem que fazer a alteração manualmente no `mainwindow.xml` .

RELATÓRIO NUTRICIONAL

cookie

Nome: cookie
Calorias: 0
PorcaoGramas: 100
GorduraTotal: 25,1
GorduraSaturada: 8
Proteina: 0
Sodio: 306
Potassio: 107
Colesterol: 0
Carboidratos: 65,3
Fibras: 2
Acucar: 33
DataValidacao: 17/05/2026 22:15:10

E com relação aos erros de conexão na service com o banco em nuvem no grupo dois foram vários primeiramente, a requisição HTTP está batendo de frente com uma barreira de segurança do .NET:

"Erro de Conexão: Erro de comunicação HTTP: The SSL connection could not be established..."

Esse erro ocorre porque o servidor onde a sua API está hospedada (runasp.net) provavelmente utiliza um certificado SSL que o seu Windows/C# não reconhece como 100% confiável (pode ser um certificado autoassinado, expirado ou gratuito).

Banana fibra

Pesquisar

Resposta da Pesquisa

Erro de Conexão: Erro de comunicação HTTP: The SSL connection could not be established, see inner exception. (The SSL connection could not be established, see inner exception.)

Gerar PDF

Alimento	Calorias	Proteína	Açúcar	Carboidratos	Gordura	Fibras
----------	----------	----------	--------	--------------	---------	--------

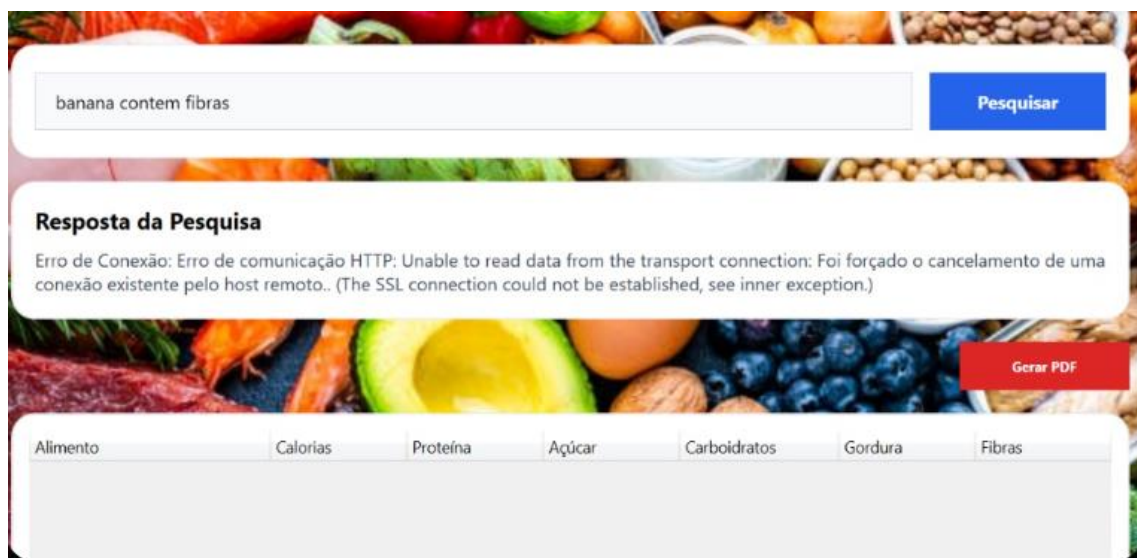
a inicialização do `_clientHandler` e do `_httpClient` em variáveis em linha (inline) podia sofrer com problemas de ordem de inicialização estática em ambientes de execução WPF.

Ao mover essa lógica para dentro de um construtor estático (`static AlimentoService()`), garantimos que a primeira vez que o programa instanciar ou chamar essa classe, o manipulador que ignora o erro de SSL será blindado e anexado ao cliente HTTP antes de qualquer tentativa de conexão.

Segundo erro do SSL persiste exatamente igual mesmo após forçar o `ServerCertificateCustomValidationCallback` como `true`, significa que o seu sistema operacional (Windows) ou o próprio ecossistema do .NET está bloqueando a conexão a nível de protocolo de transporte (TLS) antes mesmo do código do `HttpClient` conseguir interceptar o handshake.

Isso é muito comum no `runasp.net` porque ele pode estar usando uma versão de TLS muito antiga (como TLS 1.0 ou 1.1) ou uma versão que exige cifras específicas que o Windows desabilitou por segurança.

Para resolver isso de forma definitiva no WPF, precisamos forçar o aplicativo a aceitar os protocolos TLS 1.2 e TLS 1.3 globalmente e garantir o bypass.



"Unable to read data from the transport connection: Foi forçado o cancelamento de uma conexão existente pelo host remoto."

Quando o erro diz que o host remoto (o servidor da API) forçou o cancelamento, significa que o seu código C# tentou ignorar o SSL e iniciar a conversa, mas o servidor da hospedagem (runasp.net) olhou para a requisição, viu que ela não tinha uma criptografia válida ou estrita o suficiente para as regras dele, e derrubou a conexão na marra (deu um TCP Reset).

Como o servidor deles é extremamente rígido e recusa o aperto de mão (handshake) quando tentamos burlar o SSL pelo código, a única forma de resolver isso sem mexer no servidor deles é mudar a estratégia do HttpClient para usar o protocolo padrão do próprio Windows da sua máquina (que já possui as chaves de criptografia atualizadas).



"Erro ao processar os dados recebidos (JSON inválido): '<' is an invalid start of a value..."

O que significa esse erro?

O seu código C# configurou o HttpClient esperando receber uma lista de textos no formato JSON (que sempre começa com uma chave { ou colchete []).

No entanto, quando você tirou o "s" e chamou <http://apinutrifood.runasp.net/>, o servidor enviou de volta uma página de texto HTML (que começa com o caractere < de <!DOCTYPE html> ou <html>). O C# tentou ler o < achando que era um JSON e disparou esse erro.



... <title>Swagger UI</title> ...

